

Program No 13

Aim

To implement a **multi-user chat server using TCP as the transport layer protocol**, where multiple clients can communicate with the server and the server can send a manually entered reply to the particular client who sent the message.

Server Algorithm

1. Create a TCP socket.
2. Bind the socket to an IP address and port.
3. Listen for incoming client connections.
4. Accept a client connection.
5. Create a separate thread to handle the client.
6. Receive the message from that client.
7. Display the message.
8. Enter the reply manually at the server.
9. Send the reply using that client's socket.
10. Continue receiving messages from the client.
11. Close the connection when the client disconnects.

Client Algorithm

1. Create a TCP socket.
2. Connect to the server.
3. Enter a message.
4. Send the message to the server.
5. Wait for the server's reply.
6. Display the reply.
7. Repeat the process.

TCP Server

```
import socket
import threading
def handle_client(client_socket, client_address):
    print("Client connected:", client_address)
    while True:
        message = client_socket.recv(1024).decode()
        if not message:
            break
        print("\nMessage from", client_address, ":", message)
        # Server manually enters the reply
        reply = input("Enter reply: ")
```

```

        # Send reply only to this client
        client_socket.send(reply.encode())
    client_socket.close()
    print("Client disconnected:", client_address)

server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server_socket.bind(("localhost", 9999))
server_socket.listen(5)
print("TCP Server started...")
print("Waiting for clients...\n")
while True:
    client_socket, client_address = server_socket.accept()
    # Handle each client separately
    thread = threading.Thread( target=handle_client,  args=(client_socket, client_address) )
    thread.start()

```

TCP Client

```

import socket
client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client_socket.connect(("localhost", 9999))
print("Connected to server")
while True:
    message = input("Enter message: ")
    client_socket.send(message.encode())
    reply = client_socket.recv(1024).decode()
    print("Reply from server:", reply)

```

Result

The **multi-user TCP chat server** was successfully implemented. The server receives messages from multiple clients and sends the manually entered reply only to the corresponding client.